



PREMIUM CONTRACT: ИГРЫ (GAME)

Версия: 1.0.0
Дата: 2026-06-14
Статус: ☒ Complete
Доступ: Только "Шейх"

1. ОБЗОР

1.1 Назначение

Игры (Game) — это premium вложение для запуска мини-игр прямо в чате Balloo Messenger. Позволяет пользователям развлекаться, соревноваться и проводить время вместе без выхода из мессенджера.

1.2 Статус доступа

Параметр	Значение
Доступ	Premium (Шейх)
Категория	Развлечения
Сложность	Высокая
Время реализации	7-10 дней (базовые игры)
Мультиплеер	Да (2-8 игроков)

1.3 Use Cases

СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ

Развлечение в чате

"Давайте сыграем в крестики-нолики пока ждём!"

Турниры

"Еженедельный шахматный турнир команды"

Обучение

"Викторина для проверки знаний после тренинга"

Ставки

"Спорим 100 баллов на победу в нардах?"

Командообразование

2. ТИПЫ ДАННЫХ

2.1 Основной интерфейс

```
/**
 * Тип вложения игры
 */
export type GameAttachmentType = 'game';

/**
 * Категории игр
 */
export type GameCategory =
  | 'board'           // Настольные (шахматы, шашки)
  | 'card'            // Карточные (дурак, уно)
  | 'arcade'          // Аркады (змейка, тетрис)
  | 'puzzle'          // Головоломки
  | 'quiz'            // Викторины
  | 'strategy'        // Стратегии
  | 'casual'          // Казуальные
  | 'multiplayer';    // Мультиплеерные

/**
 * Статусы игры
 */
export type GameStatus =
  | 'waiting'         // Ожидание игроков
  | 'starting'        // Начало
  | 'playing'         // В процессе
  | 'paused'          // На паузе
  | 'finished'        // Завершена
  | 'abandoned'       // Покинута
  | 'cancelled';      // Отменена

/**
 * Вложение игры
 */
export interface GameAttachment {
  type: 'game';
  attachmentId: string;
  gameId: string;

  // Информация об игре
  game: GameInfo;
```

```

// Игроки
players: GamePlayer[];
maxPlayers: number;
minPlayers: number;

// Состояние
status: GameStatus;
gameState: GameState;

// Ход
currentPlayer?: string;           // userId
turnNumber: number;

// Ставки (если есть)
betting?: GameBetting;

// Результаты
results?: GameResults;

// Настройки
settings: GameSettings;

// Временные метки
createdAt: number;
startedAt?: number;
finishedAt?: number;

// Метаданные
metadata: Record<string, any>;
}

/**
 * Информация об игре
 */
export interface GameInfo {
  id: string;
  name: string;
  description: string;
  category: GameCategory;
  icon: string;
  coverImage?: string;
  developer: string;
  version: string;
  rating: number;                // 0-5
  totalGames: number;           // Сколько сыграно
  isPremium: boolean;           // Только для Шейх
  minPlayers: number;
  maxPlayers: number;
  averageDuration: number;      // секунды
  difficulty: 'easy' | 'medium' | 'hard';
  tags: string[];
}

```

```

/**
 * Игрок
 */
export interface GamePlayer {
  userId: string;
  displayName: string;
  avatar?: string;
  isSheikh: boolean;
  role: 'player' | 'spectator' | 'host';
  team?: string; // Для командных игр
  score?: number;
  rank?: number;
  joinedAt: number;
  isReady: boolean;
  isOnline: boolean;
  disconnectCount: number;
  leavingAt?: number; // Если покидает
}

/**
 * Состояние игры (абстрактное, специфично для каждой игры)
 */
export interface GameState {
  board?: any; // Для настольных
  deck?: any; // Для карточных
  entities?: any[]; // Для аркад
  questions?: QuizQuestion[]; // Для викторин
  currentPlayer?: string;
  turnTimeRemaining?: number;
  lastMove?: GameMove;
  history: GameMove[];
}

/**
 * Ход в игре
 */
export interface GameMove {
  playerId: string;
  action: string;
  data: any;
  timestamp: number;
  isValid: boolean;
  result?: string;
}

/**
 * Ставки
 */
export interface GameBetting {
  enabled: boolean;
  currency: 'virtual' | 'real';
  entryFee: number; // Взнос
  prizePool: number; // Призовой фонд
}

```

```

distribution: {                                // Распределение
  first: number;                               // % победителю
  second?: number;                             // % второму
  third?: number;                              // % третьему
};
totalBets: number;
bets: GameBet[];
}

export interface GameBet {
  playerId: string;
  amount: number;
  placedAt: number;
}

/**
 * Результаты игры
 */
export interface GameResults {
  winner?: string;                             // userId победителя
  winners?: string[];                          // Для нескольких победителей
  rankings: GameRanking[];
  duration: number;                            // Длительность в секундах
  totalMoves: number;
  statistics: Record<string, any>;
  replayUrl?: string;                          // Ссылка на повтор
}

export interface GameRanking {
  rank: number;
  playerId: string;
  score: number;
  reward?: number;                             // Выигрыш
  ratingChange?: number;                       // Изменение рейтинга
}

/**
 * Настройки игры
 */
export interface GameSettings {
  turnTimeLimit?: number;                      // Время на ход (сек)
  totalGameTime?: number;                      // Общее время игры
  allowSpectators: boolean;
  isPublic: boolean;
  isRanked: boolean;                           // Рейтинговая
  allowBetting: boolean;
  maxBet?: number;
  autoStart: boolean;
  difficulty?: 'easy' | 'medium' | 'hard';
  map?: string;                                // Для аркад
  rules?: Record<string, any>;                 // Кастомные правила
}

```

2.2 Типы игр

```
/**
 * Конкретные типы игр
 */
export type GameType =
  | 'chess'           // Шахматы
  | 'checkers'       // Шашки
  | 'backgammon'     // Нарды
  | 'tic_tac_toe'    // Крестики-нолики
  | 'durak'          // Дурак
  | 'uno'            // Уно
  | 'snakes'         // Змейка
  | 'tetris'         // Тетрис
  | 'quiz'           // Викторина
  | 'trivia'         // Тривия
  | 'poker'          // Покер
  | 'go'             // Го
  | 'reversi'        // Реверси
  | 'connect_four';  // 4 в ряд

/**
 * Конфигурация шахмат
 */
export interface ChessGame extends GameAttachment {
  game: GameInfo & { type: 'chess' };
  gameState: ChessState;
}

export interface ChessState {
  board: ChessSquare[][];
  whiteKingPos: { x: number; y: number };
  blackKingPos: { x: number; y: number };
  currentTurn: 'white' | 'black';
  castling: {
    white: { kingSide: boolean; queenSide: boolean };
    black: { kingSide: boolean; queenSide: boolean };
  };
  enPasantTarget?: { x: number; y: number };
  halfMoveClock: number;
  fullMoveNumber: number;
  isCheck: boolean;
  isCheckmate: boolean;
  isStalemate: boolean;
  capturedPieces: {
    white: ChessPiece[];
    black: ChessPiece[];
  };
  moveHistory: ChessMove[];
}
```

```

export interface ChessSquare {
  piece: ChessPiece | null;
  color: 'white' | 'black';
  isAttacked: boolean;
  isPossibleMove: boolean;
}

export interface ChessPiece {
  type: 'king' | 'queen' | 'rook' | 'bishop' | 'knight' | 'pawn';
  color: 'white' | 'black';
  hasMoved: boolean;
}

export interface ChessMove {
  notation: string; // "e4", "Nf3", "O-O"
  from: { x: number; y: number };
  to: { x: number; y: number };
  piece: ChessPiece;
  captured?: ChessPiece;
  isCheck: boolean;
  isCheckmate: boolean;
  isCastling: boolean;
  isEnPassant: boolean;
  isPromotion: boolean;
  promotionPiece?: ChessPiece;
  timestamp: number;
}

/**
 * Конфигурация крестиков-ноликов
 */
export interface TicTacToeGame extends GameAttachment {
  game: GameInfo & { type: 'tic_tac_toe' };
  gameState: TicTacToeState;
}

export interface TicTacToeState {
  board: ('X' | 'O' | null)[][];
  currentTurn: 'X' | 'O';
  winner?: 'X' | 'O' | 'draw';
  winningLine?: { x: number; y: number }[];
  moveHistory: TicTacToeMove[];
}

export interface TicTacToeMove {
  player: 'X' | 'O';
  position: { x: number; y: number };
  timestamp: number;
}

/**
 * Конфигурация викторины

```

```

*/
export interface QuizGame extends GameAttachment {
  game: GameInfo & { type: 'quiz' };
  gameState: QuizGameState;
}

export interface QuizGameState {
  currentQuestion: number;
  totalQuestions: number;
  questions: QuizQuestion[];
  answers: QuizAnswer[];
  timeRemaining: number;
  scores: Record<string, number>;
}

export interface QuizQuestion {
  id: string;
  question: string;
  options: string[];
  correctAnswer: number;
  category: string;
  difficulty: 'easy' | 'medium' | 'hard';
  points: number;
  timeLimit: number;
  image?: string;
}

export interface QuizAnswer {
  questionId: string;
  playerId: string;
  selectedAnswer: number;
  isCorrect: boolean;
  timeTaken: number;
  points: number;
}

```

2.3 Рейтинговая система

```

/**
 * Рейтинг игрока
 */
export interface PlayerRating {
  userId: string;
  gameType: GameType;
  rating: number;           // ELO рейтинг
  gamesPlayed: number;
  wins: number;
  losses: number;
  draws: number;
  winRate: number;
}

```



```

currentStreak: number;
bestStreak: number;
rank: number; // Позиция в лидерборде
tier: 'bronze' | 'silver' | 'gold' | 'platinum' | 'diamond' | 'master';
division: number; // 1-4 внутри тира
ratingHistory: RatingChange[];
lastPlayedAt: number;
}

export interface RatingChange {
  gameId: string;
  change: number; // +15, -10, etc.
  newRating: number;
  opponentRating: number;
  result: 'win' | 'loss' | 'draw';
  timestamp: number;
}

/**
 * Лидерборд
 */
export interface Leaderboard {
  gameType: GameType;
  period: 'daily' | 'weekly' | 'monthly' | 'all_time';
  entries: LeaderboardEntry[];
  lastUpdated: number;
}

export interface LeaderboardEntry {
  rank: number;
  playerId: string;
  displayName: string;
  avatar?: string;
  rating: number;
  gamesPlayed: number;
  winRate: number;
  isSheikh: boolean;
  trend: 'up' | 'down' | 'same';
}

```

3. API SPECIFICATION

3.1 Endpoints

GAME API ENDPOINTS		
POST	/api/v1/games/create	Создать игру

POST	/api/v1/games/:id/join	Присоединиться
POST	/api/v1/games/:id/leave	Покинуть игру
POST	/api/v1/games/:id/start	Начать игру
POST	/api/v1/games/:id/move	Сделать ход
POST	/api/v1/games/:id/resign	Сдаться
POST	/api/v1/games/:id/pause	Пауза
POST	/api/v1/games/:id/resume	Продолжить
GET	/api/v1/games/:id	Получить игру
GET	/api/v1/games/:id/history	История ходов
GET	/api/v1/games/list	Список игр
GET	/api/v1/games/leaderboard	Лидерборд
GET	/api/v1/games/:id/replay	Повтор игры
WS	/ws/games/:id	WebSocket для ходов

3.2 Создать игру

Endpoint: `POST /api/v1/games/create`

Headers:

```
Authorization: Bearer <token>
Content-Type: application/json
X-Sheikh-Status: true
```

Request Body:

```
{
  "gameType": "chess",
  "name": "Шахматный матч",
  "description": "Дружеская партия",
  "settings": {
    "turnTimeLimit": 300,
    "totalGameTime": 600,
    "allowSpectators": true,
    "isPublic": false,
    "isRanked": true,
    "allowBetting": false
  },
  "invitedPlayers": ["user_123", "user_456"],
  "betting": {
    "enabled": false
  }
}
```

Response (Success 201):

```
{
  "success": true,
  "data": {
    "gameId": "game_abc123",
    "attachmentId": "att_xyz789",
    "inviteUrl": "https://balloo.app/games/join/game_abc123",
    "status": "waiting",
    "players": [
      {
        "userId": "user_current",
        "displayName": "Вы",
        "role": "host"
      }
    ],
    "message": "Игра создана. Ожидание игроков..."
  }
}
```

3.3 Сделать ход

Endpoint: POST /api/v1/games/:id/move

Request Body (Шахматы):

```
{
  "move": {
    "from": { "x": 0, "y": 1 },
    "to": { "x": 0, "y": 3 },
    "piece": "pawn"
  },
  "notation": "e4"
}
```

Response:

```
{
  "success": true,
  "data": {
    "gameId": "game_abc123",
    "moveNumber": 5,
    "currentPlayer": "user_opponent",
    "gameState": {
      "board": "...",
      "isCheck": false,
      "isCheckmate": false
    },
    "timeRemaining": {
```

```
    "user_current": 450,  
    "user_opponent": 580  
  }  
}  
}
```

WebSocket Event (всем игрокам):

```
{  
  "event": "move_made",  
  "data": {  
    "gameId": "game_abc123",  
    "playerId": "user_current",  
    "move": {  
      "notation": "e4",  
      "from": { "x": 0, "y": 1 },  
      "to": { "x": 0, "y": 3 }  
    },  
    "timestamp": 1718400000000  
  }  
}
```

3.4 Лидерборд

Endpoint: GET /api/v1/games/leaderboard

Query Parameters:

```
?gameType=chess  
&period=weekly  
&limit=100  
&offset=0
```

Response:

```
{  
  "success": true,  
  "data": {  
    "gameType": "chess",  
    "period": "weekly",  
    "lastUpdated": 1718400000000,  
    "entries": [  
      {  
        "rank": 1,  
        "playerId": "user_789",  
        "displayName": "GrandMaster",  
      }  
    ]  
  }  
}
```

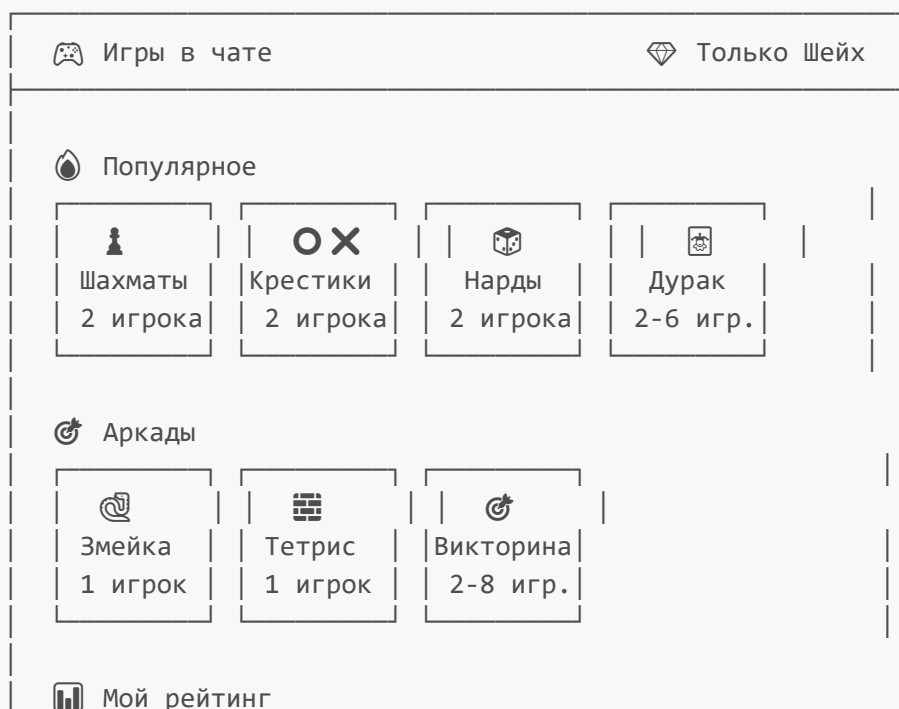
```

    "avatar": "https://...",
    "rating": 2450,
    "gamesPlayed": 150,
    "winRate": 0.72,
    "isSheikh": true,
    "trend": "up"
  },
  {
    "rank": 2,
    "playerId": "user_current",
    "displayName": "Бы",
    "avatar": "https://...",
    "rating": 2380,
    "gamesPlayed": 120,
    "winRate": 0.68,
    "isSheikh": true,
    "trend": "same"
  }
],
"userRank": {
  "rank": 2,
  "totalPlayers": 5000
}
}
}

```

4. UI/UX SPECIFICATION

4.1 Выбор игры



Шахматы: 2380 (Топ-50) ★ 68% побед

4.2 Игровое поле (Шахматы)

Шахматы

Ход белых

Чёрные (Иван)

8:30

♔

♖

♘

♚

♗

♙

♞

♜

♟

♙

♟

♙

♟

♙

♟

♙

•

•

•

•

•

•

•

•

•

•

•

•

•

•

•

•

♙

♟

♙

♟

♙

♟

♙

♟

♜

♖

♘

♚

♗

♙

♞

♜

9:45

Белые (Вы)

♚ Съедено: ♔ ♘

♜ Съедено: ♟ ♙

[Чат]

[Предложить ничью]

[Сдаться]

4.3 Результаты игры

Игра завершена!

♚

♜

ПОБЕДА

Вы победили Ивана!

PROFESSEUR : M.DA ROS

◆ 14 / 20 ◆

BTS SIO BORDEAUX - LYCÉE GUSTAVE EIFFEL

 Статистика партии:
 Длительность: 23:45
 Ходов: 47
 Рейтинг: +15 (2380 → 2395)

[ Реванш] [ Статистика] [ Поделиться]

4.4 Модальное окно (не Шейх)

 Игры доступны только Шейх

Разблокируйте доступ к мини-играм в чате!

 Доступные игры:

 Шахматы	 Нарды
 Крестики-нолики	 Дурак
 Змейка	 Викторины
 Тетрис	 И многие другие!
 Рейтинговые матчи	
 Ставки (виртуальные)	
 Турниры	

[ Отмена] [ Оформить Шейх за \$9.99/мес]

5. БИЗНЕС-ЛОГИКА

5.1 Создание игры

```
async function createGame(  
  userId: string,  
  gameType: GameType,  
  settings: GameSettings  
) : Promise<GameAttachment> {  
  
  // 1. Проверка статуса Шейх
```

```

const user = await getUserById(userId);
if (!user.is_sheikh) {
  throw new PremiumRequiredError('game');
}

// 2. Валидация типа игры
const gameConfig = GAME_CONFIGS[gameType];
if (!gameConfig) {
  throw new InvalidGameTypeError(gameType);
}

// 3. Создание игры
const game: GameAttachment = {
  type: 'game',
  attachmentId: generateAttachmentId(),
  gameId: generateGameId(),
  game: {
    id: gameType,
    name: gameConfig.name,
    description: gameConfig.description,
    category: gameConfig.category,
    // ... остальные поля
  },
  players: [{
    userId: userId,
    displayName: user.displayName,
    role: 'host',
    isReady: true,
    joinedAt: Date.now()
  }],
  minPlayers: gameConfig.minPlayers,
  maxPlayers: gameConfig.maxPlayers,
  status: 'waiting',
  gameState: initializeGameState(gameType),
  turnNumber: 0,
  settings: {
    ...gameConfig.defaultSettings,
    ...settings
  },
  createdAt: Date.now()
};

// 4. Сохранение
await saveGame(game);

// 5. Отправка приглашений
if (settings.invitedPlayers) {
  await sendGameInvitations(game.gameId, settings.invitedPlayers);
}

return game;
}

```


5.2 Валидация хода

```
async function validateMove(  
  gameId: string,  
  playerId: string,  
  move: GameMove  
): Promise<ValidationResult> {  
  
  const game = await getGame(gameId);  
  
  // 1. Проверка статуса игры  
  if (game.status !== 'playing') {  
    throw new GameNotPlayingError(game.status);  
  }  
  
  // 2. Проверка очередности  
  if (game.currentPlayer !== playerId) {  
    throw new NotYourTurnError(game.currentPlayer);  
  }  
  
  // 3. Проверка времени  
  if (game.settings.turnTimeLimit) {  
    const lastMove = game.gameState.history[game.gameState.history.length - 1];  
    const timeTaken = Date.now() - lastMove.timestamp;  
  
    if (timeTaken > game.settings.turnTimeLimit * 1000) {  
      throw new TimeExpiredError();  
    }  
  }  
  
  // 4. Валидация хода (специфично для игры)  
  const isValid = await validateGameMove(  
    game.game,  
    game.gameState,  
    move  
  );  
  
  if (!isValid) {  
    throw new InvalidMoveError(move);  
  }  
  
  return { valid: true };  
}
```

5.3 Расчёт рейтинга (ELO)

```
function calculateEloChange(  
  playerRating: number,
```

```

    opponentRating: number,
    result: 'win' | 'loss' | 'draw',
    kFactor: number = 32
  ): number {

    // Ожидаемый результат
    const expectedScore = 1 / (1 + Math.pow(10, (opponentRating - playerRating) / 400));

    // Фактический результат
    const actualScore = result === 'win' ? 1 : result === 'draw' ? 0.5 : 0;

    // Изменение рейтинга
    const ratingChange = Math.round(kFactor * (actualScore - expectedScore));

    return ratingChange;
  }

  // Пример:
  // Игрок 2380 vs 2450, победа
  // expectedScore = 1 / (1 + 10^(70/400)) = 0.401
  // ratingChange = 32 * (1 - 0.401) = +19

```

6. WebSocket REAL-TIME

6.1 Подключение

```

// Клиент
const ws = new WebSocket(`wss://balloo.app/ws/games/${gameId}`);

ws.onopen = () => {
  console.log('Connected to game');
  ws.send(JSON.stringify({
    type: 'join',
    playerId: userId,
    token: authToken
  }));
};

ws.onmessage = (event) => {
  const message = JSON.parse(event.data);
  handleGameEvent(message);
};

```

6.2 События

```
// Сервер → Клиент
type ServerEvent =
| { type: 'player_joined'; playerId: string }
| { type: 'player_left'; playerId: string }
| { type: 'game_started'; gameState: GameState }
| { type: 'move_made'; move: GameMove; playerId: string }
| { type: 'turn_changed'; currentPlayer: string }
| { type: 'time_update'; timeRemaining: Record<string, number> }
| { type: 'game_finished'; results: GameResults }
| { type: 'error'; error: GameError };

// Клиент → Сервер
type ClientEvent =
| { type: 'join' }
| { type: 'leave' }
| { type: 'ready' }
| { type: 'move'; move: GameMove }
| { type: 'resign' }
| { type: 'draw_offer' }
| { type: 'draw_accept' }
| { type: 'pause' }
| { type: 'chat'; message: string };
```

7. МЕТРИКИ

```
interface GameMetrics {
// Активность
totalGames: number;
activeGames: number;
gamesToday: number;

// Игроки
uniquePlayers: number;
averagePlayersPerGame: number;
peakConcurrent: number;

// Вовлечённость
averageGameDuration: number;
gamesPerUser: number;
retentionRate: number;

// Доходы (ставки)
totalBets: number;
totalPrizePool: number;
platformFee: number;

// Рейтинги
averageRating: number;
```

```
ratingDistribution: Record<string, number>;  
}
```

 **Статус документа:** Complete

 **Baloo - Проверни общение!**